

Introduction to Computers and Java

Primitive Data Types

Control Statements and Structured Programming

Array

Procedural Programming: Methods and APIs

Introduction to Classes, Objects, Methods

Strings and Wrapper Classes

Classes, Objects and Methods: A Deeper Look

Object-Oriented Programming: Inheritance

Object-Oriented Programming: Polymorphism

Graphical User Interface (GUI)

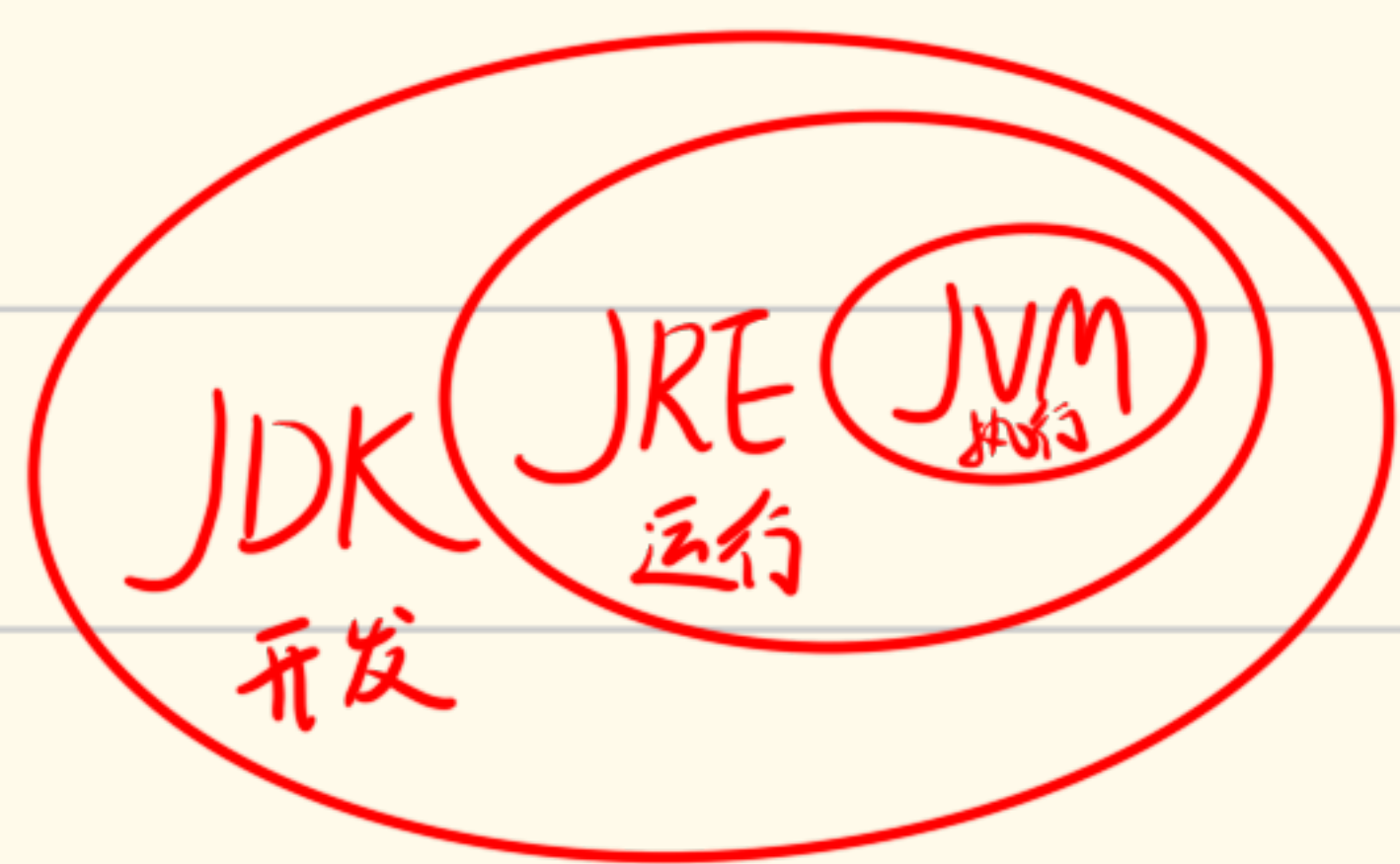
Generic Classes and Methods

Exception Handling: A Deeper Look

Arrays.toString(): 一维直接打印, 二维给各行哈希码

int... a 之类的多参数必须放在所有形参的最后

||, && 会短路(前确定后不判), 而 |, & 不会



source code <sup>compiler</sup> → bytecode  
.java .class  
↓ interpreter  
output

常见 error 类型: syntax error (语法错误).

compile error (编译错误), runtime error (运行时错误).

logic error (逻辑错误)

switch() 的控制变量可以是 byte, short, int, char 及其包装类型; enum, String; 不支持 long, float, double, boolean 及以上未提到的引用类型.

while(1) 在 Java 中不合法

循环外使用 break, 会导致 compile error

continue; 的作用是跳过循环的剩余部分直接迭代;

与所处作用域无关, 它作用于当前循环

elif 不合法

单行 if 可以不加大括号

按值传递的变量在方法内的 ++ 等不影响方法外该变量的值

String 是按值传递的 (与其不可变性有关)

当声明包装类变量时, 如果没有初始化会被赋为 null

内容相同的String和StringBuilder用equals比较会返回false.

关键字均为小写.

标识符中: a~z, A~Z, 0~9, \$, \_;

不能以数字开头

byte 8 bits, short 16 bits, int 32 bits, long 64 bits,

float 32 bits, double 64 bits, boolean 8 bits (possibly),

char 16 bits

运算符优先级: \*, /, % > +, -; 同级从左到右

else 总与最近的if关联(大括号限制除外)

空语句: 分号表示, 空体: 大括号包围而无内容; 前者表示不执行操作, 后者表示待实现

变量: 声明、赋值、使用(顺序不能颠倒)

方法中声明的变量必须在使用前赋值

for 语句中循环延续条件可以省略(始终为真)

初始值可以省略(事先声明并赋值)

迭代操作可以省略(在for内部进行)

迭代操作可包含多个表达式(逗号分隔)

do...while: 先执行后检查

格式: do { ... } while (...);

操作至少会执行一次

switch 的 case 必须接常量表达式

switch 的 case 执行: 从匹配到 break / 语句末尾

break 可选但非必需 ↑  
falling through

default 同可选: 如果无匹配和 default, 执行下一语句

break: 在 while, for, do...while, switch 中执行

continue: 跳过剩余部分, 进行下次迭代

for 循环中会先迭代后判断

算符优先级: 后自增减 > 前自增减、前缀单目运算符(+、-)、

!、类型转换 > 乘除 > 加减 > 大小比较 > 相等比较

> 布尔和 & > 布尔异或 ^ > 布尔或 | > 条件和 &&

> 布尔或 || > 三元操作符 ? : > 赋值、组合运算符

以上, 自增减、前缀单目、!、类型转换、三元、赋值、组合

结合性从右到左, 其余均为从左到右

为数组分配空间时必须给出数组大小

数组中各元素默认值: 0 (数字基本类型)、\u0000 (char)、

false (boolean)、null (reference types)

int[] n = {1, 2, 3} 的形式只允许在声明时使用 (也就是说不能分成两半)

foreach 语句仅拥有读访问权限

void 方法中可以写 return: 用于终止本方法运行并返回到调用方

`public static void main (String[] args)`: 命令行传参

parameter 是形式参数, argument 是实际参数

构造器不指定返回类型 (void 也不行)

如果类中有显式构造器, 编译器不再自动创建默认构造器

```
char[] charArray = {'h', 'e', 'l', 'l', 'o'};  
String s = new String(charArray, 3, 2); // "lo"
```

两个 String 对象直接赋值相同内容会指向同一个地址,

但通过 `new String (...)` 创建相同内容也会指向不同地址

String 的不可变性: <sup>immutability</sup>任何关于已有 String 的修改都会创建一个新的 String 对象

`s1.concat(s2)` 不会修改 `s1` 和 `s2`; `+` 也能连接字符串

StringBuilder: mutable, 初始容量 16 chars, `append (...)` 可在末尾追加多种类型的值 (基本类型、字符数组、String、Object 等)

shadow: 当局部变量与实例变量同名时, 局部变量会遮盖实例变量 (优先被访问); `this` 可以取消 shadow 从而调用实例变量

`this` 在构造函数中的使用: 必须是构造函数体的第一句

Composition: Has-a    Inheritance: Is-a

Static 变量: 在类的范围内, 意味着所有实例共享相同的变量副本 (仅一个副本)

静态方法不能访问非静态成员，归属于类本身而不是某个实例；this 在静态方法中不能使用

不要忽略 main() 方法本身是个 static method，不能访问实例方法或实例变量

toString() 不能用于 primitive types (没有方法)

String.valueOf(...): 静态方法，将其他数据类型转 String  
包的声明要在其他所有声明和语句之前

String 属于 java.lang，一个隐式导入的包

只要不导入就必须全称导入，不论事实上其他包中有没有与他同名的类

import java.util.\*; 中的 "\*" 是通配符，意为导入包中所有可用类和接口

import static ...; 导入类的静态成员，使用时不再需要类名. 方法/字段的格式

如果从不同类中导入同名静态方法，会出现编译错误

缺省修饰符 — 包访问权限

最顶层的类声明只能是 public 和 package-private (不能缺省)

一个源码文件中可以有多个类，但只有一个可以是 public; <sup>(顶层类)</sup>

其他类不能有 public 修饰符，访问权限必须是 package-private

类里可以声明另一个类，但方法里不能声明另一个方法 <sup>every</sup>

final 变量必须显式初始化，否则无法编译 (直接赋值/构造器赋值)

打印枚举类型的常量时相当于调用了toString()

方法, 输出常量名称的字符串

枚举类型构造函数默认private (不能是public); 不能使用new操作符创建枚举类型的实例  
Direct d = Direct.EAST;

枚举常量: 隐式static和隐式final

包装类: 使对应的基本数据类型 treated as objects

Integer.valueOf(...) int/String 转成 Integer

Integer.parseInt(...) Integer 转成 int

每个数字包装类都包含方法... Value()

Autoboxing 和 Autounboxing

ArrayList<T> 是泛型类, T是类型占位符 (不能装基本类型)

Heap Space: 堆空间, 在其中创建的对象具有全局访问权限(整个生命周期); 垃圾回收在堆内存上进行, 回收没有任何引用的对象

Stack Space: 栈空间, 执行线程中的方法信息、方法特定值/对堆中对象的引用); 内存相对堆空间小得多

toString() 创建堆空间中的String

java.lang.Object 是所有类的直接/间接父类

构造器不能被继承; 字段和方法能否被继承取决于其修饰符

|                                  | class | package | subclass   | world |
|----------------------------------|-------|---------|------------|-------|
| public                           | ✓     | ✓       | ✓          | ✓     |
| protected                        | ✓     | ✓       | ✓          | ✗     |
| no modifier<br>(package-private) | ✓     | ✓       | ✗ (同包子类 ✓) | ✗     |
| private                          | ✓     | ✗       | ✗          | ✗     |

super 调用父类的构造函数时必须在子类构造函数的第一行;

如果没有 super(...), 则会隐式调用父类的无参构造器

子类中的重写方法将覆盖父类中的实现, 经常被重写的方法有

toString(), equals(...) 等; 重写方法访问权限不低于父类; 返

回类型相同或为父类方法返回类型的子类型

super 的用法: ①调用父类构造函数 ②调用重写后被覆盖的父类方法 ③引向父类的实例变量

多态: 父类引用指向子类, 运行时调用的方法由被指向的对象

(子类) 决定, 子类版本被调用 (so-called dynamic binding); 但

compiler 仍认为该对象是父类类型, 因而会在父类中寻找相应的方法, 如果不存在会出现 compilation error

法, 如果不存在会出现 compilation error

为避免以上问题, 使用 downcast 转到子类类型 (一般会使用

instanceof 确保是此子类类型)

多态中父类引用不能调用子类所有成员 (访问权限)

Dynamic Binding: 运行时确定方法调用与实现 (实际类型)

Static Binding: 编译时确定方法调用与实现 (声明类型)

静态方法不能被重写, 但当子类定义同名静态方法时可以被隐藏

final 类不能被继承或扩展

抽象方法不能是 private、final 或 static (无法被子类重写)

抽象类不能用于实例化(但可以有构造函数, 作用是保证在具体的子类中总是以固定方式实例化)

接口类似于抽象类(定义类必须做什么), 但目的是为相关类或不相关的对象指定公共行为

接口中所有常量字段都是隐式的 public static final ; 接口中所有 abstract、default、static 方法都是隐式 public 的; 如果方法不是 default 或 static , 那就是隐式 abstract 的

接口可扩展多个其他接口(逗号分隔), 但不能实现其他接口

|   | Abstract Class                                                                               | Interface                                                                         |
|---|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1 | An abstract class can extend only one class or one abstract class                            | An interface can extend any number of interfaces                                  |
| 2 | An abstract class can extend another concrete class or abstract class                        | An interface cannot extend classes                                                |
| 3 | In abstract class keyword “abstract” is mandatory to declare a method as an abstract         | In an interface keyword “abstract” is optional to declare a method as an abstract |
| 4 | An abstract class can have constructors                                                      | An interface cannot have a constructor                                            |
| 5 | An abstract class can have protected and public abstract methods                             | An interface can only have public abstract methods                                |
| 6 | An abstract class can have static, final or static final variables with any access specifier | An interface can only have public static final (constant) variable                |

可以将子类对象赋值给父类变量(已实例化), 但如果反过来则必须将父类强制转换成相应子类

当子类 this 调用另一构造器时, 父类构造器只会调用一次

Array 的 toString() 和 ArrayList 的 print 都以中括号包裹  
类文件命名: 多个类并列时以 public (顶层类) 为准; 大类包  
小类时以大类为准 (即使小类中有 public)

所有类都有 toString() 方法 (继承自 Object)

一个类可以同时实现一个抽象类和多个接口, 但要注意,  
在声明时先 extends 再 implements

错误三类型: syntax, runtime, semantic

10 的算术错误 ArithmeticException 属于 Runtime Error

设 int num = 0, num += 0.1: 运算时把 num 为 double, 但最终赋值时会赋给 num (int) 结果 (强制类型转换) 所以是 0.

swap() 方法用于交换两个变量的值 (通过 temp 实现)

目前来看, 任何传基本数据类型作参的方法都不会改变  
原参数的值

变量类型: 具体类: 本类对象 / 具体子类对象

抽象类: 具体子类

接口: 有实现的对象

基本数据类型的变量储存在 stack space

抽象类可以没有抽象方法

static 方法里不能有 this 关键字

Java 中, 方法内部对引用类型的参数进行重新赋值不会  
影响外部引用, 因为传递的是引用副本而不是引用本身

如果父类引用不指向子类对象(未体现多态性),则不能将父类向下转换为子类

方法的访问权限不能低于所在类的访问权限